

Git and Github

1.What is Git

Git is a version control system.

Git helps you track changes in your code so you don't lose work and can work safely with others.

Think of Git like this

It's a time machine for your code

You can:

- save your work
- go back to old versions
- see who changed what
- work with a team without overwriting each other's code

Why Git is used

- Keep history of code changes
- Undo mistakes easily
- Work on new features without breaking main code
- Collaborate with other developers

Real-life example

Imagine you're writing notes:

- You save version 1
- Then version 2
- Then version 3

If version 3 is bad, Git lets you go back to version 2 easily.

Important Git words

- Repository (repo) → Folder where Git tracks code
- Commit → Save a snapshot of your code
- Branch → Separate line of work (experiment safely)
- Main / Master → Main working branch

Git vs GitHub

- Git → Tool (works on your computer)
- GitHub → Website to store Git projects online

Git Commands

1. **git init** is a command used to create a new Git repository by initializing Git in a project folder.

2. **git add .**

Adds all new and modified files in the current folder (and sub-folders) to the staging area.

3. **git add <file-name>**

Adds only the specified file to the staging area.

4. **git status** is a command used to show the current state of the Git repository, including tracked, untracked, staged, and modified files.

5. **git commit -m "message"** is a command used to save staged changes permanently to the Git repository with a short description (commit message).

// Download git lens it will show you tracking

6. **git diff** is a command used to show the differences between file versions, i.e., what has been changed in the code but not yet committed.

then for exist write **q** for exit

7. **git reset <file-name>** is a command used to remove the specified file from the staging area, without deleting or changing the actual file content in the working directory.

8. **Git log** is a Git command used to view the history of commits in a repository. It shows details like commit ID, author, date, and commit message for each commit.

9. **git log --oneline** is a Git command used to view the commit history in a short, single-line format, where each commit is shown with a short commit ID and its message.

10. **git reset --hard <hashcode>** is a Git command used to move HEAD and the current branch to a specific commit and permanently discard all changes and commits after it (both staged and unstaged).

11. **git revert <hashcode>** is a Git command used to undo a specific commit by creating a new commit that reverses its changes, without altering the existing commit history.

When git revert <hashcode> causes a merge conflict

This happens when the code you are reverting was changed again in later commits.

How to handle the merge conflict

1. Run revert

`git revert <hashcode>`

Git says → merge conflict

2. Check which file has conflict

`git status`

You will see something like:

both modified: one.js

3. Open the conflicted file

code one.js

Inside the file you will see:

`<<<<<< HEAD`

current code

`=====`

code from reverted commit

`>>>>>> hashcode`

4. Fix the conflict

- Keep the code you want
- Remove these lines:
 - `<<<<<<`
 - `=====`
 - `>>>>>>`
- Save the file

5. Mark conflict as resolved

`git add one.js`

6. Continue the revert

`git revert --continue`

7. Done

A new commit is created that safely undoes the old commit.

If you want to cancel revert

git revert --abort

12. **git checkout -b branch-name** is a Git command used to create a new branch and switch to it at the same time.

13. **git branch** is a Git command used to create, list, or manage branches.

A branch is a separate line of work where you can make changes without affecting the main code.

14. **git checkout master or main** is a Git command used to switch from the current branch to the master branch.

15. **git merge branch-name** is a Git command used to combine changes from one branch into another branch.

16. **git branch -D branch-name** is a Git command used to force delete a branch, even if it is not merged.

Part =2

Git rebase is a Git command used to move your commits on top of another branch's latest commits to keep the commit history clean and straight.

git rebase master means:

put your current branch's commits on top of the latest master branch commits.

git checkout Payment

git rebase master

then your merge conflict occurs and you would handle manually

then git status then git add . then git rebase --continue don't do git commit

then i button means insert mode. if you want to change the name payment vala so it is your choice

then click esc you will out from insert mode it is end.

then :wq now your work is completed

then your E also show merge conflict then do the same process again.

then git checkout master

then git merge payment now whole thing come in single line or linear

PR means pull request it will be reviewed by your senior in company then your code will merge to your main

In company never changes in your main

when your PR review is successful then you will merge your code in main .

after that you will do

`git pull origin main` fetches and merges the latest changes from the remote main branch into the current branch.

`git branch -r`

`git branch -r` is a Git command used to show all remote branches.

What it shows:

- Branches that exist on the remote repository (like origin)

origin/main

origin/dev

origin/feature-login